

AgentID — Product Requirements Document

Product: AgentID — the trust layer for agent-to-agent communication **Tagline:** TLS for agentic services

Owner: Lakshya Karira **Status:** Draft **Last updated:** July 2026

How to read this document

This PRD treats AgentID as **one product**: a trust layer that answers six questions on every agent request. Every feature is presented the same way — **the problem that forced it, why we added it, and what it does** — in the order the thinking developed. Each feature exists because the previous one exposed a gap. Nothing here is a feature for its own sake.

1. What AgentID is

AgentID sits between AI agents and the tools, APIs, and other agents they interact with, and answers six questions on every request:

1. **Who are you?** (identity)
2. **Can I trust you?** (authentication)
3. **What are you allowed to do?** (authorization)
4. **What data are you allowed to receive?** (data policy)
5. **Can I revoke you?** (lifecycle)
6. **Can I prove what happened?** (audit)

Positioning: A2A and MCP define *how* agents communicate. AgentID defines *whether they should trust each other* and *what they're allowed to do*. It doesn't replace those protocols — it runs on top of them, exactly as TLS runs on top of HTTP.

2. The root problem

AI agents today authenticate with static, long-lived API keys.

```
Agent -> static API key -> Tool
```

This one design choice creates every problem the product solves:

- A key proves the key exists — not **which organization issued the agent**, not **which agent** is calling.
- If a key leaks or an agent is compromised, there is no **fast, cross-tool way to revoke it**.
- The full payload **crosses every boundary** with no field-level control.
- Logs are **self-reported and mutable** — no independently verifiable record.
- When one company's agent calls another company's tool, there is **no cross-organization trust model at all**.

MCP and A2A standardize how agents talk and how tools are described. They deliberately leave trust, identity, authorization, revocation, and cross-org policy out of scope. That gap is the product.

3. Features — each with its problem and reasoning

3.1 Cryptographic identity (DID + Verifiable Credential)

Problem. An API key is anonymous. When a request arrives, the tool has no idea which agent sent it or which organization stands behind it. Shared or stolen keys are indistinguishable from legitimate ones.

Why we added it. Trust starts with identity. Before deciding what an agent may do, you must know *who it is* in a way that can't be forged.

What it does. Every agent gets an Ed25519 key pair and a Decentralized Identifier (DID), starting with `did:web` — resolvable over HTTPS at `/.well-known/did.json`, no blockchain required. A W3C Verifiable Credential binds the agent's DID to its owning organization and permissions, signed by AgentID. Signing keys live in AWS KMS and never enter an agent pod. The resolver is built so stronger DID methods (`did:ion`, `did:key`) can plug in later for cross-org use without changing the developer experience. The customer never sees a DID — it's an implementation detail. Decentralized identity is used only where it earns its place, not because it's fashionable.

3.2 Credential vault + dispossession (short-lived scoped tokens)

Problem. Even with an identity, if the agent still *holds* the raw API key, the whole trust layer is optional — a compromised agent or careless developer can call the tool directly and bypass everything. This is the most dangerous loophole.

Why we added it. Make bypass *impossible* rather than *forbidden*. If the agent never holds a raw, long-lived key, there is nothing to bypass with. This flips the model from "please route through us" to "we are the only source of working credentials."

What it does. Real tool credentials live server-side in a vault; agents never receive them. At call time, AgentID verifies identity, checks policy and revocation, and issues a **short-lived (default 60s), scoped** token for that specific tool and action. On expiry the agent must re-request. This mirrors HashiCorp Vault's dynamic secrets. Bonus: a memory dump of an agent or sidecar yields an expiring token, not a permanent key — shrinking the blast radius of any compromise.

3.3 Authorization / policy engine (OPA + Rego)

Problem. Knowing who an agent is doesn't tell you what it should do. A finance agent should reach Stripe, not payroll. Without central rules, permissions scatter across code and config, impossible to audit or change consistently.

Why we added it. Authorization (question 3) must be centrally defined, consistently enforced, changeable without redeploying agents — and evaluated without adding latency.

What it does. Policies in OPA/Rego support allowed tools, actions, time limits, environment restrictions, and rate limits. Compiled policy bundles are **pushed** to the enforcement point and evaluated **in-process** — no per-request network call (see section 4).

3.4 Data policy — structured field minimisation

Problem. A tool request carries the full payload. A call needing only amount and currency might drag along a customer ID, email, and SSN. Once data crosses a boundary, control is lost — especially acute in Agent-as-a-Service, where a company hands data to a third-party provider.

Why we added it. Question 4 — what data may the agent actually send? The honest claim is not "we control what the provider does with data" (impossible once they hold plaintext) but "we ensure the data never reaches them in the first place." That's an interceptor pattern, and it's sound.

What it does. A field-level allow/deny policy per agent x tool. Denied fields are stripped or masked **before the request leaves the sender's boundary** — in the sidecar, on the customer's side. The receiver never sees what was removed. Toggleable, so customers pay compute only for what they enable. This works cleanly for **structured JSON**; unstructured content is handled separately (section 3.9) and never claimed as complete redaction.

3.5 Revocation engine (Status List + signed tombstone broadcast)

Problem. When an agent is compromised, how fast can you cut its access to *every* tool? With API keys: hours to days of manual, per-service rotation. In an autonomous, high-frequency environment that window is catastrophic.

Why we added it. Instant cross-tool revocation is the most demonstrable value in the product — the "padlock moment," and question 5.

What it does. Revocation flips a bit in a VC Status List 2021 and broadcasts a **signed tombstone** via webhook fanout to every registered tool adapter — target 0.3s. Adapters without a webhook fall back to the short-TTL status list. One command: `agentid deprovision --agent <name> --all`. Two mechanisms in parallel — webhook push for near-instant, status list for everyone else — close the gap between "revoke pressed" and "access gone."

3.6 Audit — signed, tamper-evident records

Problem. After the fact, can you prove what an agent did? Standard logs live on a server, controlled by whoever runs it, and can be edited or deleted. In cross-company or regulated contexts, "trust our logs" isn't enough.

Why we added it. Question 6. Regulation (EU AI Act, SOC2 for AI, GDPR) increasingly demands verifiable audit for automated actions, and in AaaS the client needs to verify what a provider accessed independently of the provider's own records.

What it does. Every call produces an append-only record — agent DID, tool, action, fields sent/stripped, policy applied, timestamp, allow/deny — each **signed** (Ed25519). A public hash chain (each entry commits to the previous) makes tampering detectable; customers can optionally mirror the stream. The write is **async and non-blocking**. Honest framing: this is **tamper-evident**, not tamper-proof — deletion is detectable; full trustlessness needs a public ledger, which is a later, opt-in addition.

3.7 The hot path / cold path split + the sidecar

Problem. All six checks happen on *every* request. If each calls a central server, you stack multiple network round-trips onto every agent action — 100ms+ overhead. A latency-sensitive product dies here, and "TLS for agents" collapses (TLS is invisible precisely because it's nearly free).

Why we added it. To make the trust layer fast enough to be invisible, frequent work must be local and expensive coordination must be rare. This decision makes everything else viable.

What it does. - **Hot path** — runs every request, sub-10ms target: verify credential locally, evaluate cached policy in-process, check cached revocation, strip structured fields, emit audit async. Runs in a **sidecar** — a lean helper process in the same pod as the agent, reached over localhost (<1ms), doing all enforcement with no network hop. - **Cold path** — runs rarely, latency-tolerant: issuing credentials, updating policy, broadcasting revocation, storing audit. This is the Kubernetes control plane.

Key principle — push, don't pull. The cold path *pushes* policy bundles and revocation status into the hot path's local cache the moment they change, so the hot path always has the answer locally. Same trick as TLS: expensive handshake once (cold), cheap cached session thereafter (hot).

3.8 Lean-and-secure sidecar discipline

Problem. A sidecar next to every agent is multiplied thousands of times and is itself a *trust component* — a high-value target. A bloated sidecar means high cost, more attack surface, and every generic sidecar downside (CPU, memory, harder debugging, security exposure). Compromise it and an attacker could forge trust decisions.

Why we added the discipline. "Lean" (do the minimum) and "secure" (limit blast radius) are the same pressure from two directions. A small, credential-light sidecar is both fast and hard to exploit.

What it does. - **Lean:** one small compiled binary (Rust/Go), tens of MB of RAM, holding only its own credential, verification keys, the compiled policy bundle, a revocation snapshot, and a small decision cache. It does *not* issue credentials, hold long-lived signing keys, store audit logs, or run heavy models. - **Secure:** holds only short-lived tokens; minimal audited dependencies; **signed images with admission control** (cosign/Sigstore) so a tampered sidecar can't start; runs non-root, read-only filesystem, distroless (no shell); **fails closed** on the security decision but with a short-TTL cache so a brief control-plane blip doesn't deny everything; patched centrally as infrastructure with no customer code change.

Why a sidecar and not a library: a library runs inside the agent's process and can be skipped or patched by a compromised agent; a separate sidecar process can't be tampered with the same way, and paired with the egress policy (section 3.12) it becomes unavoidable rather than optional.

3.9 Content inspection for unstructured data

Problem. Field-level stripping works on structured JSON but does nothing for a scanned PDF, an image, or a free-form prompt ("summarize this document"), where sensitive data is unstructured text or pixels. For many real workloads — especially financial and legal documents — that's exactly where the sensitive data lives.

Why we added it — carefully. The capability is valuable but introduces two hazards the rest of the product lacks: it's **heavy** (OCR is CPU-intensive, a VLM is model-scale) and **probabilistic** (it can miss things). The product sells cryptographic *certainty*, so a "probably caught it" component must be walled off and honestly labelled.

What it does. A **pluggable content-inspection interface** with backends: text -> Microsoft Presidio (NER PII detection); PDF/image -> a document-AI provider (AWS Textract, Google Document AI, or a self-hosted model for customers who can't send documents out). It runs as a **separate, opt-in, independently-scaled service — never on the hot path**, invoked only when a payload is actually unstructured and policy requires inspection.

Design rules that keep it safe: we do **not** build our own OCR/VLM (it's a solved commodity and not the moat); the product is the *policy enforcement around* inspection — deciding when it's required, applying the returned redaction, refusing to forward below a configurable confidence threshold, and writing an audit-of-inspection record. Always labelled **best-effort**, never sold as a guarantee.

3.10 Deployment modes — the trust ladder the customer chooses

Problem. Different customers have different constraints. A bank asks "will you see our data?" A tiny startup asks "we don't have spare capacity to run your stack." One architecture can't serve both. And there's a deeper contradiction: in a hosted model, to inspect data the proxy must decrypt it — which means AgentID could see it, recreating the exact "why trust you?" problem.

Why we added it. The honest answer isn't one architecture — it's a **menu framed as a ladder**, where the customer picks their point on the trust-versus-effort tradeoff, and we state precisely what's true for each rung. The analogy: cash in your own safe (self-host), in a bank (hosted), or in a safe-deposit box the bank can't open (TEE) — same money, different tradeoffs.

What it does — the ladder:

Mode	Who runs it	What AgentID can see	Effort for customer	Typical customer
Self-host	Customer	Nothing — data never reaches AgentID	High (they run the stack)	Banks, regulated, security-strict
Hosted (metadata-only)	AgentID	Labels, not content — which agent, which tool, which policy, allow/deny	Low	Most companies
Edge verifier	Cloudflare	Verification only	Lowest	Browser / serverless / edge agents
Hosted + TEE	AgentID (in enclave)	Provably nothing , yet still inspects content	Low, premium cost	Regulated but can't self-host

- **Self-host** — the customer runs everything (sidecars, control plane, proxy) inside their own infrastructure. Minimisation happens on their machines; AgentID never receives their data. Trust becomes moot — there's nothing to trust us with. Strongest, simplest guarantee, shippable today.
- **Hosted (metadata-only)** — AgentID runs the stack, but sensitive data is cleaned in the sidecar on the customer's side *before* anything reaches the control plane. The control plane sees only metadata. The courier analogy: we see the envelope's address, never the letter inside.
- **Edge verifier** — for agents with no pod to attach a sidecar to (browser, serverless), the identity/policy check runs on a Cloudflare Worker a few milliseconds away, off the customer's infrastructure. Same check, different delivery.
- **Hosted + TEE** — a **Trusted Execution Environment** is a sealed region of the processor where code runs and data is processed such that even the operator (AgentID) cannot see inside. The data-inspection

proxy runs *inside* the enclave: it decrypts, inspects, and strips — all sealed. **Remote attestation** lets the hardware (AWS Nitro / Intel SGX) cryptographically prove to the customer that a genuine enclave is running exactly the published code, so the customer trusts the silicon and the math, not AgentID's word. This resolves the inspect-vs-see contradiction: inspection happens *and* exfiltration is provably impossible. Heavier, hardware-dependent, and strong-but-not-absolute (TEEs have had side-channel research), so it's a premium tier built when a real regulated-but-can't-self-host customer needs it — not a launch feature.

Baseline hygiene, stated precisely. TLS in transit and encryption at rest everywhere. But "encrypted" is **never** allowed to imply "we can't see it" in a mode where we technically can — the real non-visibility guarantee comes from self-host, metadata-only, or TEE, not from transit encryption. A sharp security buyer catches that distinction immediately; being precise is what earns trust.

Load management. The sidecar is deliberately light, so per-pod cost is small. A **DaemonSet** option runs one shared enforcement process per node — fifty agents on a machine share one helper instead of fifty. Heavy inspection (section 3.9) is a separate opt-in service the customer scales on their own terms. Principle: **customers pay compute only for the protection they switch on.**

3.11 Customer choice — expose the trust ladder, hide the security mechanics

Problem. "Let the customer choose" is two different ideas. Letting them choose their *trust posture* is right. Letting them reconfigure *how security works* is a trap: for a security product, a customer who misconfigures doesn't get a broken feature — they get an insecure system that *looks* fine, a footgun with your logo on it. Endless options also cause decision paralysis, and the optionality goes unused.

Why we added the principle. Choices about *their world* (data, tools, trust level, infrastructure) depend on their business and must be theirs. Choices about *our craft* (whether signatures are verified, whether revocation is enforced) are our expertise and must be safe by default and not disableable.

What it does. - **Customer chooses:** deployment mode (the ladder), and *what to protect* — which tools, which fields are sensitive, which agents exist. These depend on their business; we can't know them. -

AgentID decides and locks: signatures always verified, revocation always enforced, audit always written, sidecar always fails closed. No legitimate reason to disable these; every illegitimate one leads to a breach. -

Safe default with an escape hatch: data-proxy on/off, inspection confidence threshold, sidecar-per-pod vs DaemonSet — ship the safest default, let advanced users tune, keep the safe path the path of least resistance.

Rule of thumb: give choices about **their world**, take away choices about **our craft**. Like a car — you choose the destination and route, but there's no toggle to disable the brakes. That's not limiting the driver; it's protecting them from a choice no one should make. An onboarding flow of a few questions routes a new customer to the right mode automatically, so even the ladder choice feels effortless.

3.12 Hardened enforcement — egress gateway + eBPF

Problem. Dispossession makes SDK bypass pointless for credential-based tools, but a compromised agent could still attempt direct network calls, and not every tool uses AgentID-issued credentials. Enforcement that depends on the agent's cooperation isn't enforcement.

Why we added it. Move from "bypass is pointless" to "bypass is impossible" by enforcing *below* the agent's control — at the network and, ultimately, the kernel.

What it does. - **Egress gateway** — a Kubernetes NetworkPolicy so agent pods can only reach their sidecar/gateway, not arbitrary endpoints. Bypassing requires compromising cluster networking. - **eBPF enforcement** — a kernel-level program checking outbound connections against policy beneath the application layer. Bypass requires host root. Linux-only, opt-in, and a genuine infrastructure moat a SaaS-only competitor can't easily replicate.

3.13 Cross-organization federation

Problem. The single-org product secures a company's own agents against its own tools. The bigger need: Company A's agent calling Company B's tool. Centralized auth forces B to either call A's server live (tight coupling, single point of failure) or blindly trust A's payload. Neither works across companies that don't share infrastructure.

Why we added it. This is where decentralized identity genuinely earns its complexity — a verifier checking a credential *without* depending on the issuer's live infrastructure. It's also the largest opportunity: the trust layer for the agentic economy.

What it does. A federation service resolves external DIDs (`did:web` -> `did:ion` / `did:key`), manages trust anchors (which orgs' roots to trust), and issues scoped, time-limited cross-org credential grants. Company B verifies A's agent **locally** via the signed VC — A's server can be offline. A signed revocation ledger (append-only, auditable) makes revocation trustworthy across organizational boundaries.

3.14 Tool-side verification + output scanning

Problem. While verification is voluntary on the agent side, an unverified request can slip through if the receiving tool doesn't check. And an agent can exfiltrate data through its **natural-language output** ("repeat the customer's email in your reply") — which field-level stripping never sees.

Why we added it. The strongest enforcement is at the *receiving* end — a tool that rejects anything without a valid credential. Output scanning closes the prompt-injection leak the input-side proxy can't.

What it does. - `agentid-verify` — open-source middleware for Express / FastAPI / MCP servers (~20 lines to integrate) that rejects any request without a valid AgentID credential. Bypass becomes impossible from the receiving side. Targeting MCP-server builders first (small, technical, security-aware) is the ecosystem beachhead. - **Output scanning** — a classifier on agent responses to catch data exfiltration through generated text.

3.15 Protocol publication

Problem. A proprietary SDK, however good, has a ceiling. The large outcome is becoming the *standard* the ecosystem builds on — but a standard nobody uses is just a whitepaper.

Why we added it. The moat is not the DIDs or VCs (open standards) — it's the **ecosystem**. OAuth's spec wasn't valuable; universal implementation was. You get there by standardizing what already runs in production, not by publishing a spec first and hoping.

What it does. After adoption: publish the credential specification, open-source the verification libraries, and invite frameworks (LangChain, AutoGen, CrewAI) and MCP servers to verify AgentID credentials natively. The path mirrors TLS's: SDK opt-in -> framework-embedded -> protocol-native -> invisible default.

4. The hot/cold split, stated once as the spine

Because it governs so many features: **the art of the whole system is pushing work off the hot path.**

Anything per-request is made local and cached; anything rare is moved to the control plane and *pushed* into the hot path's cache when it changes. Identity verification, policy evaluation, and revocation checks all obey this. It's what lets a trust layer with six per-request checks add only single-digit milliseconds.

5. Target users and buyer

Persona	Primary need
Platform / infra engineer	Central control, instant revocation, no key sprawl
AI engineer	Drop-in SDK, minimal code change, secure by default
Security / compliance lead	Audit trail, policy enforcement, data governance
AaaS provider	Provable trust to win enterprise clients
Enterprise security architect	Cross-org agent trust without integration
Tool / MCP-server vendor	Reject unverified agents natively

Primary buyer: platform/infrastructure teams at companies running agents in production. **Budget owner:** CTO or CISO depending on company size.

6. Non-functional requirements

Category	Requirement
Latency	~5-10ms hot-path target; localhost sidecar call <1ms
Availability	Control plane 3 replicas/AZ; offline-first local verification
Scalability	Sidecar-per-agent scales by construction; revocation service autoscaled; DaemonSet option to amortise
Security	KMS/HSM keys; short-lived tokens; signed images + admission control; distroless, non-root, read-only FS; fail-closed with short-TTL cache
Encryption	TLS in transit + at rest baseline; self-host / metadata-only / TEE for the trust guarantee
Reliability	Async non-blocking audit; graceful degradation
Observability	Tracing, metrics, structured logs on control plane
Compatibility	Every capability is additive; earlier deployments keep working unchanged

7. Tech stack

Layer	Technology
Sidecar (hot path)	Rust or Go, distroless
Edge verifier	Cloudflare Workers + KV
Identity	Ed25519, did:web (-> did:ion/did:key for cross-org), W3C VC Data Model
Policy engine	OPA / Rego, in-process on hot path
Revocation	VC Status List 2021, webhook fanout, Redis pub/sub
Structured data proxy	Field-level JSON stripping
Content inspection	Microsoft Presidio, AWS Textract / Google Document AI (pluggable)
Cache	Redis (sidecar-local)
Control plane	FastAPI / Go, PostgreSQL per service
Key management	AWS KMS / HSM
Orchestration	Kubernetes, rolling updates, HPA
Egress / kernel	NetworkPolicy, eBPF
Confidential compute	AWS Nitro Enclaves / Intel SGX + attestation
Image security	cosign / Sigstore, admission control
Audit	Append-only store + public hash chain

8. Success metrics

Core launch criteria - Register an agent and issue a verifiable credential in under 5 minutes from install. - Deprovision an agent across N tools in under 1 second (measured, not assumed). - Hot-path overhead measured within the single-digit-ms target under load. - Zero raw tool credentials present in any agent environment. - Every tool call produces a verifiable signed audit record.

Adoption signals - Agents registered; tools connected per agent; weekly active developers. - Self-host vs hosted split; time-to-first-revocation (activation). - First cross-org interaction with no callback; tool vendors running `agentid-verify`; frameworks with native support.

9. Honest limitations (stated as part of the pitch)

Limitation	The honest position
Latency / revocation figures	Design targets until benchmarked on a real build — labelled as such everywhere.
Unstructured redaction	Best-effort , probabilistic; configurable thresholds; never a guarantee.
Hosted data visibility	We see metadata, not content ; full non-visibility requires self-host or TEE.
Audit tamper-resistance	Tamper-evident , not tamper-proof, until a public ledger is added.
SDK bypass	Solved by dispossession + egress + tool-side verify — layered, not one silver bullet.
We are a trust chokepoint	Answered by self-host and open-sourcing the enforcement path, not by asking for blind trust.
TEE	Strong but not absolute (side-channel research exists); heavier and hardware-dependent — a premium tier, not a default.

Documenting these openly is deliberate — infrastructure earns trust by naming its boundaries, the way serious security companies publish their threat models.

10. Open questions

- Flagship tool-vendor design partner for native verification?
- Default deployment mode for first customers — self-host or hosted?
- Pricing model — per agent, per tool connection, per seat, or platform fee? (*Biggest lever; decide before launch.*)
- Does the data-proxy stage ship on or off by default?
- Sidecar-per-pod vs DaemonSet as the recommended default?
- Build a self-hosted document-inspection model, or rely solely on third-party document-AI providers?

11. Positioning one-liners

- **Vision:** "TLS for agentic services."
- **vs MCP/A2A:** "They define how agents communicate. We define whether they should trust each other."
- **vs firewalls (ArmorIQ):** "A firewall can't prove who sent the traffic. We can."
- **On trust:** "Self-host and we never see a byte; hosted, we see metadata, not content; enclave, we can't see it even if we wanted to."
- **On Web3:** "DIDs/VCs only where they earn it — cross-org trust. The customer never sees a DID."
- **On choice:** "You choose how much to trust us and what to protect. We decide how the cryptography works — and it's never disableable."
- **On the moat:** "The standard becomes ours by adoption, the way OAuth's did. The spec was never the value — the ecosystem was."

Design discipline: AgentID is one product — a trust layer answering six questions on every agent request. The hot path stays lean forever; everything expensive is opt-in and independently scaled; the customer chooses their trust posture while the security mechanics stay locked and safe by default. That discipline is what keeps this a coherent trust layer rather than a sprawling security suite.